

ISLAMIC UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering

Software Maintenance Lab Project Report

Math Problem Solver

Course Code:	SWE-4802
Course Title:	Software Maintenance Lab
Name:	Muhammad Yasir Raiyan
ID:	210042152
Dept:	CSE
Prog:	B.Sc. in S.W.E
Sem:	8th
Academic Year:	2024–25

Software Maintenance Project

2026

Contents

1	Project Overview	2
1.1	Introduction	2
1.2	Project Objectives	3
2	Initial System – V1 Baseline	4
2.1	System Description	4
2.2	Initial Project Structure	4
2.3	Baseline Testing	5
3	Corrective Maintenance	6
3.1	Scenario	6
3.2	Change ID	6
3.3	Program Comprehension	6
3.4	Change Management	7
3.4.1	Problem	7
3.4.2	Requested Change	7
3.4.3	Change Implemented	7
3.5	Impact Analysis	7
3.6	Reverse Engineering	7
3.7	Refactoring	8
3.8	Testing Result	8
3.9	Version Transition	8
4	Adaptive Maintenance	9
4.1	Scenario	9
4.2	Change ID	9
4.3	Program Comprehension	9
4.4	Change Management	10
4.4.1	Requirement	10
4.4.2	Change Implemented	10
4.4.3	Technology Added	10

4.5	Impact Analysis	10
4.6	Reverse Engineering	11
4.7	Refactoring	12
4.8	Version Transition	12
5	Preventive Maintenance	13
5.1	Scenario	13
5.2	Change ID	13
5.3	Program Comprehension	13
5.4	Change Management	13
5.4.1	Objective	13
5.4.2	Changes Implemented	14
5.5	Impact Analysis	14
5.6	Reverse Engineering	14
5.7	Refactoring	15
5.8	Result	15
5.9	Version Transition	15
6	Perfective Maintenance	16
6.1	Scenario	16
6.2	Change ID	16
6.3	Program Comprehension	16
6.4	Change Management	17
6.4.1	User Requirements	17
6.4.2	Objective	17
6.5	Impact Analysis	17
6.6	Reverse Engineering	17
6.7	Refactoring	18
6.7.1	Factorial	18
6.7.2	Percentage	18
6.8	Result	18
6.9	Testing Result	19
6.10	Version Transition	19
7	Overall Maintenance History	20
8	Testing Results	22
8.1	Testing Summary	22
8.2	Final Test Command	22
8.3	Final Test Result	22

9	Git Maintenance History	23
10	Comparison of Maintenance Categories	24
11	Conclusion	25

Chapter 1

Project Overview

1.1 Introduction

Math Problem Solver is a Python-based mathematical calculation application. The initial version of the application provides basic and advanced mathematical operations through a command-line interface. The system was subsequently adapted and improved through different software maintenance activities.

The main objective of this project is to simulate the four major categories of software maintenance:

1. Corrective Maintenance
2. Adaptive Maintenance
3. Preventive Maintenance
4. Perfective Maintenance

For each maintenance category, the following activities were performed:

1. Program Comprehension
2. Change Management
3. Impact Analysis
4. Reverse Engineering
5. Refactoring

The project demonstrates how an existing software system can be understood, modified, improved, tested, and evolved throughout its maintenance lifecycle.

1.2 Project Objectives

The major objectives of this project are:

- Identify and correct existing software defects.
- Adapt the existing system to a new execution environment.
- Improve maintainability and prevent future defects.
- Add new functionality based on user requirements.
- Analyze the impact of software changes.
- Perform reverse engineering of existing functionality.
- Refactor existing code without changing intended behavior.
- Maintain regression testing throughout the maintenance lifecycle.

Chapter 2

Initial System – V1 Baseline

2.1 System Description

The initial version of the Math Problem Solver provides mathematical calculation functionality through a command-line interface.

The baseline system contains:

- Basic arithmetic operations
- Advanced mathematical operations
- Input validation
- Result formatting
- Logging
- Unit testing
- Integration testing

2.2 Initial Project Structure

The initial project structure is shown below:

```
1 Math-Problem-Solver/  
2 |  
3 +-- app.py  
4 +-- main.py  
5 |  
6 +-- calculator/  
7 |   +-- basic.py  
8 |   +-- advanced.py
```

```
9 |    +-- validator.py
10 |
11 +-- utils/
12 |    +-- logger.py
13 |    +-- formatter.py
14 |
15 +-- tests/
16 |    +-- test_basic.py
17 |    +-- test_advanced.py
18 |    +-- test_validator.py
19 |    +-- test_integration.py
20 |
21 +-- templates/
22 |    +-- index.html
23 |
24 +-- requirements.txt
25 +-- README.md
```

2.3 Baseline Testing

Before performing maintenance activities, the initial version was tested.

Ran 10 tests

OK

The successful baseline test result was used as the reference point for subsequent maintenance activities.

Chapter 3

Corrective Maintenance

3.1 Scenario

A defect was identified in the division operation. When the denominator was zero, the application produced a `ZeroDivisionError`, which caused the application to terminate unexpectedly.

3.2 Change ID

CM-001

3.3 Program Comprehension

The existing calculation flow was analyzed to locate the source of the problem.

```
User
|
v
main.py
|
v
calculator/basic.py
|
v
divide(a, b)
|
v
a / b
```

The `divide()` function was identified as the source of the division-by-zero defect.

3.4 Change Management

3.4.1 Problem

Division by zero caused the application to crash with a `ZeroDivisionError`.

3.4.2 Requested Change

The system should prevent division by zero and provide a meaningful error message to the user.

3.4.3 Change Implemented

The `divide()` function was modified to validate the denominator before performing the calculation.

3.5 Impact Analysis

The following components were affected:

Component	Impact
<code>calculator/basic.py</code>	Direct Impact
<code>main.py</code>	Indirect Impact
<code>tests/test_basic.py</code>	Regression Test

Table 3.1: Corrective Maintenance Impact Analysis

3.6 Reverse Engineering

The original execution flow was reconstructed as follows:

```
User
|
v
Select Division
|
v
main.py
|
```

```
    v
divide(a, b)
    |
    v
Denominator = 0
    |
    v
ZeroDivisionError
```

This analysis helped identify where the defect occurred and which components required modification.

3.7 Refactoring

The division function was changed to:

```
1 def divide(a: float, b: float) -> float:
2     if b == 0:
3         raise ValueError("Cannot divide by zero")
4     return a / b
```

A regression test was added:

```
1 def test_divide_by_zero(self):
2     with self.assertRaises(ValueError):
3         divide(10, 0)
```

3.8 Testing Result

After implementing the corrective maintenance:

```
Ran 11 tests
OK
```

3.9 Version Transition

V1 → Corrective Maintenance → V1.1

Chapter 4

Adaptive Maintenance

4.1 Scenario

The original application was command-line based. The system needed to be adapted so that users could access the calculator through a web browser.

4.2 Change ID

CM-002

4.3 Program Comprehension

The existing architecture was analyzed:

```
User
  |
  v
main.py
  |
  v
Calculator Modules
  |
  v
Result
```

The calculation modules were already separated from the main application logic. Therefore, they could be reused when adapting the system to a web environment.

4.4 Change Management

4.4.1 Requirement

The existing command-line application needed to be adapted to a web-based environment.

4.4.2 Change Implemented

A Flask web interface was introduced while reusing the existing calculator modules.

4.4.3 Technology Added

- Flask
- HTML
- Web interface

4.5 Impact Analysis

The affected components were:

Component	Impact
<code>app.py</code>	Web Application
<code>requirements.txt</code>	Flask Depen- dency
<code>templates/index.html</code>	Web Interface

Table 4.1: Adaptive Maintenance Impact Analysis

The following components were reused:

- `calculator/basic.py`
- `calculator/advanced.py`
- `calculator/validator.py`

4.6 Reverse Engineering

The original flow was:

```
CLI
|
v
main.py
|
v
Validation
|
v
Calculator Function
|
v
Result
```

The adapted flow became:

```
Browser
|
v
index.html
|
v
app.py
|
v
Validation
|
v
Calculator Function
|
v
Result
|
v
Browser
```

4.7 Refactoring

A Flask web interface was added without rewriting the existing calculator logic.

The existing calculator functions were reused by `app.py`.

```
Browser
|
v
Flask app.py
|
v
calculator/
|-- basic.py
|-- advanced.py
+-- validator.py
|
v
Result
```

The Math Problem Solver can now be accessed through a web browser.

4.8 Version Transition

V1.1 → Adaptive Maintenance → V1.2

Chapter 5

Preventive Maintenance

5.1 Scenario

The application was working correctly, but improvements were required to prevent future defects and make the system easier to maintain.

5.2 Change ID

CM-003

5.3 Program Comprehension

The existing project structure and dependencies were analyzed.

The following areas were identified for improvement:

- Input validation
- Error handling
- Code readability
- Type information
- Test coverage

5.4 Change Management

5.4.1 Objective

The objective was to improve the maintainability, reliability, and readability of the system.

5.4.2 Changes Implemented

The following improvements were performed:

- Improved input validation
- Added type hints
- Improved error handling
- Added mathematical input validation
- Added additional tests

5.5 Impact Analysis

The affected components were:

Component	Impact
calculator/validator.py	High
calculator/basic.py	Medium
calculator/advanced.py	Medium
tests/test_validator.py	High
tests/test_advanced.py	High

Table 5.1: Preventive Maintenance Impact Analysis

The intended functionality of the calculator was preserved.

5.6 Reverse Engineering

The validation flow was analyzed:

```
User Input
|
v
validate_number()
|
v
Convert to number
```

```
|  
v  
Calculator  
|  
v  
Result
```

Potential invalid inputs were identified before they could cause unexpected behavior.

5.7 Refactoring

The validation function was improved with type hints and better exception handling:

```
1 def validate_number(value: str) -> float:  
2     try:  
3         return float(value)  
4     except (ValueError, TypeError):  
5         raise ValueError(  
6             "Invalid number. Please enter a valid number."  
7         )
```

The advanced calculator was also improved to handle invalid square-root and factorial inputs.

Additional validation tests were added.

5.8 Result

The software became easier to understand, maintain, and extend while preserving its existing behavior.

5.9 Version Transition

V1.2 → Preventive Maintenance → V1.3

Chapter 6

Perfective Maintenance

6.1 Scenario

Users requested additional mathematical functionality and an improved web experience.

6.2 Change ID

CM-004

6.3 Program Comprehension

The existing advanced calculator and web application were analyzed to determine where the new features should be integrated.

Browser

|

v

app.py

|

v

Calculator Module

|

v

Result

6.4 Change Management

6.4.1 User Requirements

Users requested:

- Percentage calculation
- Factorial calculation
- Improved operation selection

6.4.2 Objective

The objective was to improve the functionality and user experience of the application.

6.5 Impact Analysis

The following components were affected:

- `calculator/advanced.py`
- `app.py`
- `templates/index.html`
- `tests/test_advanced.py`

Existing basic calculator functionality remained unchanged.

6.6 Reverse Engineering

The existing feature flow was analyzed:

Browser

|

v

app.py

|

v

Operation Selection

|

v

Calculator Function

|

v

Result

The new features were integrated into the existing architecture.

6.7 Refactoring

Two new mathematical operations were added.

6.7.1 Factorial

```
1 def factorial(n: int) -> int:
2     if n < 0:
3         raise ValueError(
4             "Factorial is not defined for negative numbers."
5         )
6
7     if not float(n).is_integer():
8         raise ValueError(
9             "Factorial requires a whole number."
10        )
11
12    return math.factorial(int(n))
```

6.7.2 Percentage

```
1 def percentage(value: float, percent: float) -> float:
2     return (value * percent) / 100
```

The web interface was updated to allow users to select the new operations.

6.8 Result

The final application supports:

- Addition
- Subtraction

- Multiplication
- Division
- Power
- Square Root
- Cube Root
- Factorial
- Percentage

6.9 Testing Result

After perfective maintenance:

Ran 16 tests

OK

6.10 Version Transition

V1.3 → Perfective Maintenance → V2.0

Chapter 7

Overall Maintenance History

The complete maintenance lifecycle is shown below:

Initial V1 Baseline

|

v

Corrective Maintenance

Fix division by zero

|

v

V1.1

|

v

Adaptive Maintenance

Add Flask web interface

|

v

V1.2

|

v

Preventive Maintenance

Improve validation and code quality

|

v

V1.3

|

v

Perfective Maintenance

Add percentage and factorial

|
v
V2.0

The project evolved incrementally from the initial baseline to the final maintained version.

Chapter 8

Testing Results

Testing was performed throughout the software maintenance lifecycle to ensure that modifications did not break existing functionality.

8.1 Testing Summary

Stage	Result	Status
Initial V1 Baseline	10 tests passed	PASS
Corrective Maintenance	11 tests passed	PASS
Preventive Maintenance	Tests passed	PASS
Perfective Maintenance	16 tests passed	PASS

Table 8.1: Testing Results

8.2 Final Test Command

The final test suite was executed using:

```
1 python -m unittest discover -s tests -p "test_*.py"
```

8.3 Final Test Result

Ran 16 tests

OK

The successful execution of all 16 tests demonstrates that the final system maintained its expected functionality after the maintenance activities.

Chapter 9

Git Maintenance History

Git was used to organize the software maintenance activities into separate logical commits.

The maintenance history was organized as follows:

Initial V1 baseline

|

v

Corrective maintenance:

fix division by zero

|

v

Adaptive maintenance:

add Flask web interface

|

v

Preventive maintenance:

improve validation and code quality

|

v

Perfective maintenance:

add percentage and factorial

|

v

Complete software maintenance documentation

This commit-based organization makes it easier to identify, track, and review each maintenance activity independently.

Chapter 10

Comparison of Maintenance Categories

Category	Purpose	Project Change
Corrective	Fix existing defects	Fixed division by zero
Adaptive	Adapt to a new environment	Added Flask web interface
Preventive	Prevent future problems	Improved validation, type hints, error handling, and testing
Perfective	Improve functionality	Added factorial and percentage

Table 10.1: Comparison of Maintenance Categories

Chapter 11

Conclusion

The Math Problem Solver was successfully used to simulate the four major categories of software maintenance.

Corrective maintenance fixed an existing division-by-zero defect. Adaptive maintenance adapted the application from a command-line environment to a web-based environment.

Preventive maintenance improved validation, error handling, type hints, and testing. Finally, perfective maintenance enhanced the application by adding factorial and percentage calculations.

The maintenance activities were performed systematically using program comprehension, change management, impact analysis, reverse engineering, and refactoring.

The final system successfully passed:

16 Tests – OK

Therefore, the project demonstrates a complete software maintenance lifecycle covering corrective, adaptive, preventive, and perfective maintenance activities.